

Sentiment Analysis — Product Reviews

Evelin João Vítor Araújo Jade Colpani Clayton Liberato Raffael Paiva

Problema

Reviews de clientes em plataformas de e-commerce contém informação valiosa sobre a experiência do consumidor, mas o volume de texto torna a leitura manual inviável em escala.

Motivação

Automatizar a leitura de sentimento permite priorizar produtos e atendimentos com maior insatisfação, sem depender de revisão humana review a review.

Papel da IA na solução

Um classificador binário (regressão logística implementada em PyTorch) aprende, a partir de um vetor Bag-of-Words do texto, a distinguir reviews positivos de negativos.

Público-alvo

Equipes de e-commerce e atendimento que precisam monitorar a percepção dos clientes sobre produtos a partir de reviews de texto.

Objetivo principal

Classificar automaticamente o sentimento de um review como **positivo** ou **negativo**.

Repositório: github.com/joaovgaraujo/project-sentiment-analysis

Dados e solução proposta

Base de dados

Amazon Product Reviews — Kaggle (yasserh/amazon-product-reviews-dataset), licença CC0-1.0.

Colunas utilizadas

`reviews.text` — texto do review
`reviews.rating` — nota de 1 a 5

Variável prevista

Positivo (1) — nota ≥ 4

Negativo (0)

— nota ≤ 2

Nota 3 (neutra) é descartada

Entrada e saída do sistema

Entrada: texto bruto do review.

Saída: rótulo binário — 0 (negativo) ou 1 (positivo).

Solução proposta (pipeline)

carregar CSV → limpar/normalizar texto → converter nota em rótulo → dividir treino/teste → vetorizar (Bag-of-Words) → treinar regressão logística (PyTorch) → avaliar → salvar modelo e métricas

Requisitos funcionais

ID	Requisito funcional (origem: Decision/Question Goal — GR4ML)	Status
RF-01	O sistema deve classificar o sentimento (positivo/negativo) de cada avaliação, a partir do texto vetorizado em Bag-of-Words, para o Gerente de CX priorizar resposta.	Implementado
RF-02	O sistema deve gerar diariamente a lista de avaliações negativas, agrupada por produto e ordenada por recência, para o Gerente de Produto escalar problemas.	Implementado
RF-03	O sistema deve calcular e persistir accuracy, precisão, recall, F1 e macro-F1 a cada treinamento, para monitorar a qualidade do modelo.	Implementado

Requisitos não funcionais e de dados

ID	Requisito não funcional	Status verificado contra o código
RNF-01	Recall da classe negativa $\geq 0,85$.	Não verificável — métricas só cobrem a classe positiva; provável que NÃO seja atendido.
RNF-02	Explicabilidade via pesos do modelo linear.	Estruturalmente possível (nn.Linear expõe pesos); sem extração/relatório implementado.
RNF-03	Lote diário <15min em CPU comum; treino reproduzível.	Reprodutibilidade atendida (seed fixa); tempo de processamento não medido.

Requisitos de dados (RD)

- RD-01 — coletar reviews.text/reviews.rating (Kaggle), validar colunas. → Implementado (load_data / validate_columns)
- RD-02 — limpar texto, rotular binário, vetorizar BoW só com o treino. → Implementado (clean_text / normalize_label / build_vocabulary)
- RD-03 — monitorar distribuição de classes e rebalancear se razão > 3:1. → Não implementado (nenhuma checagem de balanceamento no código)

Critério de aceitação (negócio): sucesso se o tempo médio de resposta a negativas cair de 72h para <24h em 3 meses, e a proporção mensal de negativos cair de ~8% para ~6,4% em 6 meses. Meta de produção, ainda não medida.

Diagrama da arquitetura



O diagrama reflete o pipeline real implementado em `main.py`, orquestrado por `run_training()` em `src/training/train.py`. Cada caixa corresponde a um módulo/função existente no repositório.

Decisões arquiteturais

Responsabilidade de cada componente

- data/loader.py — carrega e valida o CSV
- preprocessing/transform.py — limpa texto, normaliza rótulos, vetoriza (BoW)
- models/model.py — define, treina e salva o modelo PyTorch
- training/train.py — orquestra split, treino, avaliação, persistência
- evaluation/metrics.py — calcula métricas com NumPy puro

Fluxo e localização

- Pré-processamento: src/preprocessing/ (funções puras)
- Treinamento: models/model.py :: train_model
- Inferência: models/model.py :: predict
- Modelo salvo em: results/model.pt (state_dict)

Comunicação entre componentes

Funções puras com contratos tipados (DataFrame → DataFrame → np.ndarray → Tensor), sem estado global compartilhado.

Por que essa arquitetura

Separação de responsabilidades
um módulo por etapa do pipeline.

Alta coesão
funções do mesmo domínio agrupadas.

Baixo acoplamento
comunicação por assinaturas tipadas.

Testabilidade
cada função testável isoladamente (44 testes).

Estrutura do projeto

```
project-sentiment-analysis/  
├── main.py  
├── requirements.txt  
├── data/  
│   ├── raw/  
│   └── processed/  
├── results/  
│   ├── metrics/  
│   └── figures/  
├── docs/  
│   └── Roteiro_GR4ML_Preenchido.docx  
├── notebooks/  
│   └── requisitos_e_objetivos.ipynb  
├── tests/  
│   ├── test_data_loader.py  
│   ├── test_preprocessing.py  
│   ├── test_model.py  
│   ├── test_predict.py  
│   ├── test_evaluation.py  
│   ├── test_split.py  
│   ├── test_properties.py  
│   ├── test_integration.py  
│   └── test_reporting.py  
└── src/  
    ├── data/loader.py  
    ├── preprocessing/transform.py  
    ├── models/model.py  
    ├── training/train.py  
    ├── evaluation/metrics.py  
    ├── reporting/negatives.py  
    ├── reporting/history.py  
    └── utils/config.py
```

Por que essa separação

O código é dividido por **estágio do pipeline** (dados → pré-processamento → modelo → treino → avaliação), e não por tipo técnico genérico.

Isso permite que cada etapa evolua e seja testada de forma independente — por exemplo, trocar a vetorização (BoW → embeddings) sem tocar no modelo ou no treino.

`main.py` não contém lógica de negócio: apenas orquestra as chamadas de alto nível, mantendo o ponto de entrada enxuto.

`tests/` espelha a estrutura de `src/`, um arquivo de teste por módulo.

Responsabilidades dos módulos

Módulo	Responsabilidade
data/loader.py	Carregamento do CSV e validação de colunas obrigatórias
preprocessing/transform.py	Limpeza de texto, normalização de rótulos, vocabulário e vetorização (BoW)
models/model.py	Definição do modelo PyTorch, treino, predição e salvamento
training/train.py	Divisão treino/teste e orquestração do pipeline completo
evaluation/metrics.py	Cálculo de métricas (accuracy, F1, precision, recall) com NumPy
utils/config.py	Constantes globais (colunas, thresholds, seed, epochs, learning rate)
tests/	44 testes automatizados (unittest) — unitários, de propriedade e de integração
docs/ · notebooks/	Caderno GR4ML preenchido e notebook de requisitos e objetivos (Entrega 4)
reporting/negatives.py · reporting/history.py	Relatório diário de negativas por produto/recência (result-YYYYMMDD.csv) e histórico versionado de métricas com macro-F1

main.py integra os componentes chamando, em sequência: `load_data()` → `preprocess_dataset()` → `run_training()` → `print_report()`.

Código Python, NumPy e tipagem

```
def load_data(path: str) -> pd.DataFrame:
    if not os.path.isfile(path):
        raise FileNotFoundError(
            f"Dataset file not found: {path}"
        )
    df = pd.read_csv(path)
    validate_columns(
        df, [TEXT_COLUMN, RATING_COLUMN]
    )
    if len(df) == 0:
        raise ValueError(
            "Dataset is empty: no rows found."
        )
    return df
```

Entrada: caminho do CSV. **Saída:** DataFrame validado.

```
def texts_to_matrix(
    texts: pd.Series,
    vocab: dict[str, int]
) -> np.ndarray:
    matrix = np.zeros(
        (len(texts), len(vocab)),
        dtype=np.float64
    )
    for i, text in enumerate(texts):
        for word in text.split():
            if word in vocab:
                matrix[i, vocab[word]] += 1
    return matrix
```

Entrada: textos + vocabulário. **Saída:** matriz Bag-of-Words (NumPy).

Ambas as funções usam **type hints** completos (parâmetros e retorno), evidenciando interfaces claras e responsabilidades bem definidas — padrão seguido em todo o `src/`.

Implementação do modelo de IA em PyTorch

```
class SentimentClassifier(nn.Module):  
    """Logistic regression: a single  
    linear layer that outputs one logit."""  
  
    def __init__(self, input_dim: int) -> None:  
        super().__init__()  
        self.linear = nn.Linear(input_dim, 1)  
  
    def forward(  
        self, x: torch.Tensor  
    ) -> torch.Tensor:  
        return self.linear(x).squeeze(1)
```

Arquitetura	1 camada linear (regressão logística)
Função de perda	BCEWithLogitsLoss
Otimizador	SGD, lr = 0,1
Épocas	100
Batch size	full-batch (sem mini-batches)
CPU/GPU	agnóstico via DEVICE (usa GPU se disponível)
Salvamento	save_model() → torch.save(state_dict)

 **Precisão importante:** não existe uma função `load_model()` reutilizável em `src/` — mas o recarregamento é demonstrado diretamente em `test_model.py` via `load_state_dict(torch.load(...))`. Funciona, mas não está encapsulado como função de produção.

Configuração experimental

Divisão dos dados

80% treino / 20% teste, embaralhamento via `numpy.random.default_rng`, seed fixa (`RANDOM_SEED = 42`) — `split_dataset()`.

Métricas utilizadas

- Acurácia
- F1-score
- Precisão
- Recall

Todas calculadas com NumPy puro, sem scikit-learn (evaluation/metrics.py).

Hiperparâmetros

- Épocas: 100
- Learning rate: 0,1
- Otimizador: SGD
- Features: Bag-of-Words (contagem de palavras)

O que o grupo buscava descobrir

Se um baseline simples — Bag-of-Words + regressão logística — já é suficiente para separar bem reviews positivos de negativos, antes de investir em representações mais custosas. Até o momento, apenas essa configuração foi documentada.

Exemplos de testes e evidências

```
class TestCleanText(unittest.TestCase):

    def test_removes_punctuation(self) -> None:
        self.assertEqual(
            clean_text("Hello, World!"),
            "hello world"
        )
        self.assertEqual(
            clean_text("don't stop!"),
            "dont stop"
        )

class TestNormalizeLabel(unittest.TestCase):

    def test_neutral(self) -> None:
        self.assertIsNone(normalize_label(3))

class TestLoadData(unittest.TestCase):

    def test_raises_file_not_found(self) -> None:
        with self.assertRaises(FileNotFoundError):
            load_data("nonexistent/path/to/file.csv")
```

Evidência real de execução

Rodados nesta preparação (módulos sem dependência de PyTorch no ambiente local):

```
$ python -m unittest tests.test_preprocessing \
    tests.test_data_loader tests.test_evaluation -v

..... (18 tests)

-----
Ran 18 tests in 0.008s

OK
```

O README do repositório documenta a suíte completa: **44 testes**, via unittest puro (sem pytest, sem Hypothesis — requirements.txt só tem pandas, numpy e torch) —
python -m unittest discover -s tests.

Resultados, conclusão e limitações

92,4%

ACURÁCIA

0,960

F1-SCORE

92,4%

PRECISÃO

100%

RECALL

Valores documentados no README após execução completa do pipeline (100 épocas).

O modelo resolveu o problema?

Não, na métrica que importa pro negócio. O recall de 100% é da classe **positiva** — mas o que RNF-01 exige é recall da classe **negativa** $\geq 0,85$, e o metrics.py atual nem calcula isso separado por classe. O padrão observado (recall positivo perfeito, dataset provavelmente desbalanceado) sugere viés para prever "positivo" — o oposto do que o Gerente de CX precisa: achar os negativos.

Requisitos atendidos?

RF-01 e RD-01/RD-02 implementados. RD-03 ainda não têm código. RNF-01 não é verificável com as métricas atuais; RNF-03 tem reprodutibilidade mas não medição de tempo.

Principais limitações

- Bag-of-Words não captura ordem ou contexto das palavras
- Sem load_model() em src/ (reload só demonstrado em teste)
- Recall só é calculado para a classe positiva — RNF-01 não é verificável hoje
- RF-02 (lista diária por produto) e RD-03 (rebalanceamento) não implementados

Próximos passos

- Calcular recall por classe e macro-F1; checar matriz de confusão
- Se confirmado o desbalanceamento: class weights ou undersampling (RD-03)